

安装必需的软件

本书中使用的所有 Python 软件都是免费的、开源的。以下各节提供有关在何处查找软件的指南，以及安装相关软件说明的超链接。由于软件总是在变化中，而 Python 的版本和本文中使用的某些模块肯定也会发生变化，因此我们不会试图为读者提供在每个操作系统和平台上安装 Python 的具体说明。

A.1 安装 Python

Python 官网 (www.python.org) 提供 Python 的最新版本。单击“Download”（下载）超链接，遵循有关操作系统的安装说明，安装最新版本的 Python 程序。

1. Windows

如果读者计算机上运行的是 Windows 操作系统，则需要下载并安装 Python。下载安装文件后，双击该文件以执行该文件。我们建议选中更新路径（诸如“Add python to PATH”）的复选框，以便 Python 更新路径并接受长文件名。

2. Mac OS X

如果读者使用的计算机是 Mac，并且使用的是 OS X 10.5 或者更高版本的操作系统，那么读者的计算机上已经安装了较旧版本的 Python (2.x)。但是，本文中的许多示例程序与此旧版本不兼容。强烈建议读者安装 Python 3.8 或者更高版本。但是，不要卸载早期版本，因为一些苹果和第三方软件使用早期版本。

3. Linux

如果读者的计算机上运行的是 Linux 系统，那么也很有可能已经安装了 Python。通过在命令行中键入以下命令可以查看安装的版本：

```
python3 -V
```

如果读者的 Linux 机器上没有安装 Python 3.x，那么需要下载并予以安装。

A.2 安装 Python 图像库和 cImage

第 6 章中使用的图像库依赖于两个 Python 模块。图像处理需要的第一个模块是 `cImage.py`。可以从 <http://knuth.luther.edu/~bmiller/python.html> 处获取此模块。单击该链接可跳转到 `cImage` 的 GitHub 存储库。

或者，读者可以直接打开 GitHub 网站 ([GitHub.com](https://github.com)) 并搜索 `cImage`。如果出现多个搜索结果，请选择 `bnmnetp/cImage`。下载压缩文件，并解压缩，然后将 `cImage.py` 移动到 `site-packages` 文件夹中。在 Windows 系统中，`site-packages` 位于 Python 安装文件夹下的 `Lib` 文件夹中。如果找不到该文件夹，只需将 `cImage.py` 移动到包含第 6 章代码的工作文件夹中。Python 可以在这两个位置的任意一处找到 `cImage.py`。`cImage` 模块可以在不需要额外的模块下处理 `gif` 格式和 `ppm` 格式图像文件。

需要安装的第二个模块称为 Python 图像库。`Pillow` 允许处理 `jpg` 格式和其他几种图像格式的图像文件。为了安装 `Pillow`，可以在命令行中键入如下命令：

```
pip install Pillow
```

Python 快速参考

本附录中的 Python 快速参考概述了本文中使用的 Python 结构。它不是一个详尽的参考，而是各种表和语法概要。这些内容在正文章节中有更详细的描述。此外，可以在 <https://docs.python.org> 上找到更详细的文档。

B.1 Python 保留关键字

表 B-1 中显示了 Python 的保留关键字。读者不要在程序中使用这些关键字作为标识符，否则将导致 `SyntaxError`。

表 B-1 Python 保留关键字

<code>and</code>	<code>class</code>	<code>except</code>	<code>global</code>	<code>None</code>	<code>return</code>
<code>as</code>	<code>continue</code>	<code>exec</code>	<code>if</code>	<code>nonlocal</code>	<code>True</code>
<code>assert</code>	<code>def</code>	<code>False</code>	<code>import</code>	<code>not</code>	<code>try</code>
<code>async</code>	<code>del</code>	<code>finally</code>	<code>in</code>	<code>or</code>	<code>while</code>
<code>await</code>	<code>elif</code>	<code>for</code>	<code>is</code>	<code>pass</code>	<code>with</code>
<code>break</code>	<code>else</code>	<code>from</code>	<code>lambda</code>	<code>raise</code>	<code>yield</code>

B.2 数值数据类型

Python 包括以下三种数值数据类型：

- 整数。
- 浮点数。
- 复数。

表 B-2 列举了数值数据类型常用的运算符

表 B-2 Python 语言中的算术运算符

运算符名称	运算符	说明
加法	<code>+</code>	计算两个值的和
减法	<code>-</code>	计算两个值的差
乘法	<code>*</code>	计算两个值的乘积
除法	<code>/</code>	计算两个值的商
整数除法	<code>//</code>	计算两个整数的商的整数部分
取模（取余数）	<code>%</code>	计算除法后的余数
乘幂	<code>**</code>	计算一个数的乘幂。例如 $(x ** y)$ 等价于 x^y

B.3 内置函数

Python 包含许多内置函数，用户可以在没有对象引用的情况下调用这些函数。表 B-3 列

举了一些常见的内置函数。

表 B-3 内置函数

函数	说明
<code>chr(num)</code>	返回 Unicode 值 <code>num</code> 表示的字符
<code>dir()</code>	返回当前局部名称空间中定义的名称
<code>float(numOrString)</code>	把值或者字符串转换为浮点数
<code>help(str)</code>	返回 <code>str</code> 的帮助文档
<code>input(prompt)</code>	输出提示信息 <code>prompt</code> ，然后读取一行文本内容并作为字符串返回
<code>isinstance(object, class)</code>	如果 <code>object</code> 是 <code>class</code> 的实例对象，则返回 <code>True</code> ；否则返回 <code>False</code>
<code>int(numOrString)</code>	把数值或者字符串转换为整数
<code>len(sequence)</code>	返回序列中的元素个数
<code>list(sequence)</code>	使用序列中的元素创建一个列表
<code>max(sequence)</code>	返回序列中的最大数据项
<code>min(sequence)</code>	返回序列中的最小数据项
<code>next(iterator)</code>	返回迭代器中的下一个数据项
<code>open(filename, mode)</code>	打开名为 <code>'filename'</code> 的文件，打开模式 <code>mode</code> 可以取以下值： <code>'r'</code> （读取）、 <code>'w'</code> （写入）或者 <code>'a'</code> （附加）
<code>ord(ch)</code>	返回单个字符的字符串的 Unicode 编码
<code>print(value1, ..., separator=None, end=endstring, file=None, flush=False)</code>	输出各值，默认使用空格分隔，默认使用换行符结束
<code>round(num, nDigits)</code>	返回四舍五入到小数点后 <code>nDigits</code> 的 <code>num</code> ；如果未指定 <code>nDigits</code> ，那么返回最接近的整数
<code>@staticmethod</code>	函数装饰器，把方法转换为静态方法
<code>str(num)</code>	把数值转换为字符串
<code>sum(sequence)</code>	返回数值序列中所有数据项的累加和
<code>super()</code>	调用方法的超类版本

B.4 序列运算符

Python 包括三种序列数据类型：字符串、列表和元组。表 B-4 总结了适用于所有序列的运算符。

表 B-4 Python 中序列的运算符

运算名称	运算符	说明
索引	<code>[]</code>	访问序列中的元素
拼接	<code>+</code>	把两个序列拼接在一起
重复	<code>*</code>	重复拼接多次
成员	<code>in</code>	判断某个数据项是否在序列中存在
成员	<code>not in</code>	判断某个数据项是否不在序列中存在
长度	<code>len</code>	返回序列中数据项的个数
切片	<code>[:]</code>	提取序列的一部分

B.4.1 字符串

字符串是由整数索引的不可变字符序列，索引从 0 开始。创建字符串字面量的方式如下所示，我们可以将其括在单引号、双引号或者三引号中：

```
a = "Don't say Hello."
b = 'She said, "Hello World!'"
c = """He said, "You Don't say!", didn't he?"""
```

表 B-5 总结了字符串的常用方法。

表 B-5 字符串方法一览

方法	使用方法	说明
center	<code>aString.center(w)</code>	返回长度为 <code>w</code> 的字符串，如果字符串长度不足 <code>w</code> ，则在该字符串的两端加相同数量的空格
count	<code>aString.count(item)</code>	返回 <code>item</code> 在 <code>aString</code> 中出现的次数
ljust	<code>aString.ljust(w)</code>	返回左对齐的长度为 <code>w</code> 的字符串，如果字符串长度不足 <code>w</code> ，则在该字符串的后面加空格
rjust	<code>aString.rjust(w)</code>	返回右对齐的长度为 <code>w</code> 的字符串，如果字符串长度不足 <code>w</code> ，则在该字符串的前面加空格
upper	<code>aString.upper()</code>	将 <code>aString</code> 中的字符全部转换为大写字母后返回
lower	<code>aString.lower()</code>	将 <code>aString</code> 中的字符全部转换为小写字母后返回
index	<code>aString.index(item)</code>	返回字符串 <code>item</code> 在字符串 <code>aString</code> 中第一次出现的索引位置，如果不存在，则返回错误
rindex	<code>aString.rindex(item)</code>	返回字符串 <code>item</code> 在字符串 <code>aString</code> 中最后一次出现的索引位置，如果不存在，则返回错误
find	<code>aString.find(item)</code>	返回字符串 <code>item</code> 在字符串 <code>aString</code> 中第一次出现的索引位置，如果不存在，则返回 <code>-1</code>
rfind	<code>aString.rfind(item)</code>	返回字符串 <code>item</code> 在字符串 <code>aString</code> 中最后一次出现的索引位置，如果不存在，则返回 <code>-1</code>
replace	<code>aString.replace(old, new)</code>	返回一个字符串，结果为字符串 <code>aString</code> 中的所有子字符串 <code>old</code> 都被替换为子字符串 <code>new</code>
split	<code>aString.split(sChar)</code>	返回在 <code>sChar</code> 处拆分后的子字符串列表

B.4.2 列表

列表是可以引用任意对象的可变序列。列表索引是从 0 开始的整数。创建列表的方式如下所示：

```
>>> emptyList = []
>>> myList = [1, 2, 3.0, 'hello', 5, 6]
>>> newList = list("hello")
>>> newList
['h', 'e', 'l', 'l', 'o']
```

表 B-6 总结了最常用的列表方法。我们可以使用 `list` 方法将序列转换为列表。

表 B-6 Python 中列表提供的常用方法

方法名称	使用方法	说明
append	<code>aList.append(item)</code>	添加一个新的数据项 <code>item</code> 到列表的末尾
insert	<code>aList.insert(i, item)</code>	把一个数据项 <code>item</code> 插入到列表的第 <code>i</code> 个位置

(续)

方法名称	使用方法	说明
pop	<code>aList.pop()</code>	移除并返回列表中的最后一个数据项
pop	<code>aList.pop(i)</code>	移除并返回列表中的第 <i>i</i> 个数据项
sort	<code>aList.sort(key=None, reverse=False)</code>	更改列表为从低到高排序状态。可选的 <code>key</code> 参数指定一个函数，用于抽取一个字段作为排序值。如果 <code>reverse</code> 参数为 <code>True</code> ，则列表从高到低排序
reverse	<code>aList.reverse()</code>	反转列表中各数据项的顺序
index	<code>aList.index(item)</code>	返回数据项 <code>item</code> 在列表中第一次出现的索引位置
count	<code>aList.count(item)</code>	返回数据项 <code>item</code> 在列表中出现的次数
remove	<code>aList.remove(item)</code>	移除列表中第一次出现的数据项 <code>item</code>

我们也可以使用以下 `range` 序列创建序列。

- `range(stop)`：创建一个序列，从 0 开始直到 `stop-1`。
- `range(start, stop)`：创建一个序列，从 `start` 开始直到 `stop-1`。
- `range(start, stop, step)`：创建一个序列，从 `start` 开始直到 `stop-1`，步长为 `step`。

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(10, 0, -1))
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

B.4.3 元组

元组是可以引用任意对象的不可变序列。创建元组的方式如下所示：

```
>>> t = (1, 2, 3)
>>> t
(1, 2, 3)
>>> u = ('a', 'b')
>>> u
('a', 'b')
>>> y = tuple([1, 2, 3])
>>> y
(1, 2, 3)
>>> z = tuple('abc')
>>> z
('a', 'b', 'c')
```

元组支持与列表相同的运算符，例如索引和切片。主要区别在于，不能以修改元组的方式使用运算符。不能将任何 `list` 方法与元组一起使用。

`zip` 函数可以用于从两个或者多个序列中创建元组列表。元组由每个序列的第 *i* 个元素组成。例如：

```
>>> list(zip('hello', range(len('hello'))))
[('h', 0), ('e', 1), ('l', 2), ('l', 3), ('o', 4)]
>>> list(zip([1, 2, 3], [4, 5, 6]))
[(1, 4), (2, 5), (3, 6)]
>>> list(zip('abc', 'def', 'ghi'))
[('a', 'd', 'g'), ('b', 'e', 'h'), ('c', 'f', 'i')]
```

B.5 字典

字典是 `key:objects` (键:对象) 的集合。对象由其键引用。这些元素按照添加到字典中的顺序进行维护。创建 Python 字典的方式如下所示:

```
>>> emptyDict = {}
>>> myD = { 'a':123, 'b':789, 'c':246 }
>>> myD['d'] = 359
>>> myD
{'a': 123, 'b': 789, 'c': 246, 'd': 359}
```

表 B-7 总结了 Python 中提供的字典方法。

表 B-7 Python 中字典提供的方法

方法名称	使用方法	说明
<code>keys</code>	<code>aDict.keys()</code>	返回字典中的 <code>dict_keys</code> 对象 (键)
<code>values</code>	<code>aDict.values()</code>	返回字典中的 <code>dict_values</code> 对象 (值)
<code>items</code>	<code>aDict.items()</code>	返回字典中的 <code>dict_items</code> 对象 (键-值元组)
<code>get</code>	<code>aDict.get(k)</code>	返回与键 <code>k</code> 相关联的值。如果键 <code>k</code> 在字典中不存在, 则返回 <code>None</code>
<code>get</code>	<code>aDict.get(k, alt)</code>	返回与键 <code>k</code> 相关联的值。如果键 <code>k</code> 在字典中不存在, 则返回 <code>alt</code>
<code>setdefault(key, default)</code>	<code>value = nDict.setdefault(key, default)</code>	如果键 <code>key</code> 在字典中不存在, 则插入值为 <code>default</code> 的键 <code>key</code> , 并返回 <code>default</code> 。如果键 <code>key</code> 在字典中存在, 则返回键 <code>key</code> 相对应的值
<code>in</code>	<code>key in aDict</code>	如果键 <code>k</code> 在字典中存在, 则返回 <code>True</code> ; 否则返回 <code>False</code>
<code>not in</code>	<code>key not in aDict</code>	如果键 <code>k</code> 在字典中不存在, 则返回 <code>True</code> ; 否则返回 <code>False</code>
<code>index</code>	<code>aDict[key]</code>	返回与键 <code>key</code> 相关联的值
<code>clear()</code>	<code>aDict.clear()</code>	清除字典中的所有数据项
<code>del</code>	<code>del aDict[key]</code>	从字典中删除一个数据项

B.6 文件

文件是存储在磁盘上的字符序列。将文件视为文本行序列也很有用。表 B-8 总结了 `File` 提供的方法。

表 B-8 文件对象提供的方法

方法名称	使用方法	说明
<code>open(filename, mode)</code>	<code>with open(filename, 'r') as fileVar</code>	打开名为 <code>filename</code> 的文件用于读取, 结果将返回指向文件对象的一个引用 (<code>fileVar</code>)。其他文件打开模式包括: 'w' (写入到文件)、'a' (附加到文件)。读取 ('r') 是默认模式。使用 <code>with</code> 方式打开文件, 当处理结束后文件将自动关闭
<code>close</code>	<code>fileVar.close()</code>	文件使用完成
<code>write</code>	<code>nChars = fileVar.write(aString)</code>	把 <code>aString</code> 添加到文件末尾。filevar 必须指向以写入方式打开的文件。返回写入的字符数
<code>read(n)</code>	<code>aString = fileVar.read(n)</code>	读取并返回包含 <code>n</code> 个字符的字符串, 如果没有指定参数 <code>n</code> , 则读取整个文件到一个字符串中
<code>readline(n)</code>	<code>aString = fileVar.readline(n)</code>	返回文件的下一个文本行, 所有文本包括换行符。如果指定参数 <code>n</code> , 并且如果文本行的长度大于 <code>n</code> , 则只返回 <code>n</code> 个字符。到达文件结尾时返回空字符串
<code>readlines(n)</code>	<code>stringList = fileVar.readlines(n)</code>	返回包含 <code>n</code> 个字符串的列表, 每个字符串表示文件的一个文本行。如果没有指定参数 <code>n</code> , 则返回文件中的所有文本行

下面的程序打开一个文件，逐行读取并打印文件的每一行内容：

```
with open('myfile.dat', 'r') as f:
    line = f.readline()
    while line:
        print(line)
        line = f.readline()
# f 自动被关闭
```

也可以使用简单的 `for` 循环读取和打印文件内容：

```
with open('myfile.dat') as f:
    for line in f:
        print(line)
```

B.7 格式化输出

Python 允许我们使用字符串 `format` 方法格式化字符串。例如：

```
>>> print("{0} + {1} = {2}".format(5, 6, 11))
5 + 6 = 11
>>> city = "Orlando"
>>> rainfall = 9.5
>>> print( "{0:>12s} {1:5.1f} inches\n".format(city, rainfall))
Orlando    9.5 inches
```

`format` 方法使用占位符指定的格式替换字符串中占位符的替换字段。默认情况下，替换字段的替换顺序与占位符相同。但是，建议使用大括号 `{}` 中的序列号来标识每个占位符，如前面代码中的 `{0}`、`{1}` 和 `{2}` 所示。

占位符的格式为：

```
{Position:AlignmentWidthPrecisionType}
```

表 B-9 列举了可用于指定值类型的各种格式化字符。

表 B-9 常用类型规格

替换字段的类型	转换字符类型	输出格式
字符串	s	字符串。这是默认格式
整型	c	字符。把整数转换为对应的 Unicode 编码
	d	十进制整数。这是整数的默认格式
	n	数值。与十进制整数相同
浮点数	e 或 E	科学计数法，默认精度为 6 位。例如 <code>m.ddddde+/-xx</code> 或者 <code>m.dddddE+/-xx</code>
	f 或 F	定点数，默认精度为 6 位。例如 <code>m.dddddd</code>
	g	通用格式。根据数值的大小，自动使用科学计数法或者定点数格式
	n	与 <code>g</code> 相同
	%	百分比。把数值乘以 100，然后显示为 <code>f</code> 格式并显示一个百分比符号 <code>%</code>

表 B-10 列举了可用于指定值的对齐、宽度和精度的各种格式化字符。

表 B-10 常用格式修饰符

修饰符类型	修饰符字符	输出格式
对齐	<	在宽度范围左对齐值。这是字符串的默认对齐方式
	>	在宽度范围右对齐值。这是数值的默认对齐方式
	^	在宽度范围居中对齐值

(续)

修饰符类型	修饰符字符	输出格式
宽度	w	值将占用 w 个字符宽度。默认为显示值的最小宽度
精度	.n	显示小数点后 n 位数字

B.8 迭代

B.8.1 数据集合上的简单迭代

for 语句允许我们使用以下格式轻松迭代任何数据集合：

```
for item in sequence:
    语句 1
    语句 2
    ...
```

以下的三个示例演示了如何在字符串、列表和元组上进行迭代。

在本例中，变量 i 将每次通过循环绑定到 range 函数创建的一个新整数：

```
for i in range(n):
    print(i)
```

在本例中，循环变量 i 每次通过循环都将绑定到列表中的下一个元素：

```
for i in ['a', 1, 'b', 2, 'c', 3]:
    print(i)
```

在本例中，循环变量 ch 被绑定到字符串的每个字符：

```
for ch in "hello world":
    print(ch)
```

也可以在字典上迭代。下面的例子演示了两种常见的模式。在本例中，循环变量 i 绑定到字典 myDictionary 中包含的每个键，并打印与每个键相关联的值。

```
for i in myDictionary:
    print(myDictionary[i])
```

还可以使用以下代码同时迭代键和值：

```
for key, value in myDictionary.items():
    print(key, value)
```

在本例中，使用 items 方法提前从字典中取出“键 - 值”对。

B.8.2 使用 while 的迭代

while 循环是一个不定循环。它将迭代执行循环体中的语句，直到循环条件变为 False。如果循环条件从不为 False，则循环是一个无限死循环，并将永远持续下去。

```
while < 循环条件 >:
    语句 1
    语句 2
    ...
    更新循环条件语句
```

B.8.3 列表解析

列表解析允许通过在列表创建表达式中嵌入 for 循环来创建列表。它比编写 for 循环

和使用 `append` 方法更加方便。以下是列表解析的一般格式：

```
[< 表达式 > for <item1> in < 序列 1>
    for <item2> in < 序列 2>
    ...
    if < 条件 >]
```

`if` 语句是可选的，至少需要一个 `for` 循环。下面是一些简单的列表解析示例：

```
>>> x = [1, 2, 3]
>>> y = ['a', 'b', 'c']

>>> [a * a for a in x]    # 各元素的平方
[1, 4, 9]
>>> [a for a in x if a % 2 != 0] # 奇数元素
[1, 3]
>>> [(a, b) for a in x for b in y] # 元组，对应于 x 和 y 的元素
[(1, 'a'), (1, 'b'), (1, 'c'),
 (2, 'a'), (2, 'b'), (2, 'c'),
 (3, 'a'), (3, 'b'), (3, 'c')]
```

B.9 布尔表达式

B.9.1 关系表达式

简单的布尔表达式可以使用表 B-11 中所示的关系运算符来构建。

表 B-11 关系运算符及其含义

关系运算符	含义
<code><</code>	小于
<code><=</code>	小于或等于
<code>></code>	大于
<code>>=</code>	大于或等于
<code>==</code>	等于
<code>!=</code>	不等于

B.9.2 复合布尔表达式

可以使用逻辑运算符 `and`、`or` 和 `not` 将多个布尔表达式连接起来，如表 B-12 所示。在该表中，`x` 和 `y` 为 `True` 或者 `False`。

表 B-12 逻辑运算符及其含义

逻辑运算符	含义
<code>x and y</code>	如果 <code>x</code> 为 <code>False</code> ，则返回 <code>x</code> ；否则返回 <code>y</code>
<code>x or y</code>	如果 <code>x</code> 为 <code>False</code> ，则返回 <code>y</code> ；否则返回 <code>x</code>
<code>not x</code>	如果 <code>x</code> 为 <code>False</code> ，则返回 <code>True</code> ；否则返回 <code>False</code>

B.10 选择结构

B.10.1 双分支选择结构

```
if < 条件 >:
```

```

    < 语句 > #< 条件 > 为 True 时执行
else:
    < 语句 > #< 条件 > 为 False 时执行

```

B.10.2 单分支选择结构

```

if < 条件 >:
    < 语句 > #< 条件 > 为 True 时执行

```

B.10.3 使用 `elif` 的嵌套选择结构

```

if < 条件 1 >:
    < 语句 > #< 条件 1 > 为 True
elif < 条件 2 >:
    < 语句 > #< 条件 1 > 为 False; < 条件 2 > 为 True
elif < 条件 3 >:
    < 语句 > #< 条件 1、2 > 为 False; < 条件 3 > 为 True
else:
    < 语句 > #< 条件 1、2、3 > 为 False

```

B.11 Python 模块

B.11.1 数学

表 B-13 总结了本书使用的 `math` 模块中的主要函数。

表 B-13 简单的数学函数

方法名称	使用方法	说明
<code>cos</code>	<code>math.cos(x)</code>	返回 x (单位为弧度) 的余弦值
<code>sin</code>	<code>math.sin(x)</code>	返回 x (单位为弧度) 的正弦值
<code>tan</code>	<code>math.tan(x)</code>	返回 x (单位为弧度) 的正切值
<code>dist</code>	<code>math.dist(p1, p2)</code>	返回两个点 <code>p1</code> 和 <code>p2</code> 之间的欧几里得距离 (指定为元组)
<code>degrees</code>	<code>math.degrees(r)</code>	把弧度转换为角度
<code>radians</code>	<code>math.radians(d)</code>	把角度转换为弧度
<code>sqrt</code>	<code>math.sqrt(x)</code>	返回 x 的平方根

B.11.2 随机数

表 B-14 总结了本书中使用的 `random` 模块中的主要函数。

表 B-14 随机数生成

方法名称	使用方法	说明
<code>random</code>	<code>random.random()</code>	返回 0.0 到 1.0 之间的一个随机数
<code>randint</code>	<code>random.randint(1, 3)</code>	返回 1 到 3 之间的一个随机整数 (包含端点)
<code>randrange</code>	<code>random.randrange(4)</code>	返回 0 到 3 之间的一个随机数。语法同 <code>range</code> 函数, 用于指定随机数范围

B.11.3 统计函数

表 B-15 总结了 `statistics` 模块中的一些实用函数。

表 B-15 `statistics` 模块中的一些实用函数

方法	说明
<code>mean(data)</code>	返回数据集的平均值
<code>median(data)</code>	返回有序数值数据的中值（中间位置的值）。如果数据项的个数为偶数，则返回中间位置两个值的平均值
<code>multimode(data)</code>	返回所有众数的列表
<code>mode(data)</code>	返回出现次数最多的值（众数）。如果存在多个出现次数一样最多的值，则返回列表中的第一个众数
<code>stdev(data)</code>	返回标准差
<code>stdev(data, mean)</code>	返回标准差，给出平均值

B.11.4 读取 CSV 文件

表 B-16 总结了 `csv` 模块中的函数，`reader` 函数用于读取 `.CSV` 文件，`next` 函数可用于任何迭代器。

表 B-16 读取 `.csv` 文件的实用函数

方法	说明
<code>reader(file)</code>	返回遍历一个 <code>.CSV</code> 文件行的迭代器对象。每一行返回为其字段的列表
<code>next(iterator)</code>	返回迭代器中的下一个数据项

B.11.5 打开 URL

表 B-17 总结了 `urllib.request` 模块中的 `urlopen` 函数，用于打开一个网址。

表 B-17 `urllib.request` 模块中的 `urlopen` 函数

方法	说明
<code>urlopen(url)</code>	打开一个 Web 网址并返回一个文件句柄。如果失败，则抛出一个 <code>URLError</code> 异常

B.11.6 读取 JSON 数据

表 B-18 总结了 `json` 模块中的 `loads` 函数，用于把 JSON 数据转换为 Python 数据格式。

表 B-18 `json` 模块中的 `loads` 函数

方法	说明
<code>loads(jsonData)</code>	把 JSON 对象转换为 Python 对象。JSON 对象转换为 Python 字典；JSON 数组转换为 Python 列表；JSON 数据类型转换为 Python 数据类型

B.11.7 抽象类和方法

表 B-19 显示了抽象类 `ABC`，表 B-20 显示了用于把方法转换为抽象方法的装饰器。

表 B-19 `abc` 模块中的 `ABC` 类

类	说明
<code>ABC</code>	抽象基类，用于方便定义抽象类。抽象类指定 <code>ABC</code> 为其超类

表 B-20 abc 模块中的 abstractmethod 装饰器

装饰器	说明
@abstractmethod	把方法转换为抽象方法的装饰器

B.11.8 多线程

threading 模块为定义多线程应用程序提供支持。多线程应用程序需要执行以下三个操作：

- 1) 类应该继承自 Thread 类。示例：`class MyApp(Thread)`。
- 2) 类应该定义一个 run 方法，该方法包含需要执行的线程的代码。
- 3) 类应该调用 start 方法。它将创建一个线程并调用 run 方法。

表 B-21 显示了 run 和 start 方法。

表 B-21 Thread 类中的 run 和 start 方法

方法	说明
run()	包含执行线程工作的代码的方法。子类包含该方法的自定义实现版本
start()	创建线程并调用线程的 run 方法。每个线程只能调用该方法一次

B.12 正则表达式

一些简单的正则表达式模式如下所示：

.	匹配任意单个字符
[abc]	匹配 a、b 或者 c
[^abc]	匹配 a、b 或者 c 之外的任意字符
[abc]+	匹配 abc 中字符一次或者多次。例如匹配 b 或者 aba 或者 ccba
[abc]+[de]*	匹配 abcde 中字符零次或者多次。例如匹配 b 或者 aba 或者 bd 或者 abae
(regex)	创建一个捕获组
\$	在字符串末尾应用模式

表 B-22 总结了 re 模块中关于正则表达式的主要函数

表 B-22 正则表达式模块函数

方法名称	使用方法	说明
match(pattern, string)	<code>re.match('[abc]XY.', 'aXYv')</code>	匹配以 a、b 或者 c 开头、后跟 XY、再接任意字符的任何字符串。如果匹配成功，返回匹配对象；否则返回 None
sub(pattern, replacement, string)	<code>re.sub('[tv]', 'X', 'vxyzbgtt')</code>	返回 'XxyzbgXX'。与 replace 类似，区别在于使用正则表达式匹配
findall(pattern, string)	<code>{re.findall('[bc]+', 'abcdefedcba')}</code>	返回 ['bc', 'cb']。返回所有与正则表达式匹配的子字符串
groups()	<code>matchObj.groups()</code>	返回匹配的所有捕获组的元组。matchObj（匹配对象）是通过调用 match 创建的
group(i)	<code>matchObj.group(i)</code>	返回匹配对象位于索引位置 i 处的捕获组的元组。matchObj（匹配对象）是通过调用 match 创建的

B.13 定义函数

```
def functionName(param1, param2, ...):
    statement1
    statement2
    ...
```

B.14 定义类

```
class classname:
    def method1()
        ...

    def method2()
        ...
    ...
```

B.15 删除对象

`del` 语句用于删除对象，例如变量、函数和可变序列中的数据项。

删除函数：

```
>>> def bar():
        return 2 * 5
>>> bar()
10
>>> del bar      # 删除 bar 函数
>>> bar()        # 尝试调用已经被删除的函数
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    bar()
NameError: name 'bar' is not defined
```

删除对象：

```
>>> import turtle
>>> t = turtle.Turtle()
>>> t.forward(50)
>>> del t        # 删除 turtle 对象
>>> t.forward(75) # 尝试使用已经被删除的 turtle 对象
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    t.forward(75)
NameError: name 't' is not defined
```

删除列表数据项：

```
>>> myList = ['a', 'b', 'c', 'd']
>>> myList
['a', 'b', 'c', 'd']
>>> del myList[2] # 删除索引位置 2 处的数据项
>>> myList
['a', 'b', 'd']
```

删除字典项：

```
>>> myDict = { 'a':123, 'b':456, 'c':789 }
>>> myDict
{'a': 123, 'b': 456, 'c': 789}
>>> del myDict['b'] # 删除键 b 的数据项
>>> myDict
{'a': 123, 'c': 789}
```

B.16 常见错误信息

学习如何阅读和理解错误消息可以节省很多时间和麻烦，因为错误消息可以帮助我们找到错误的根源。

最常见的错误之一是语法错误，它表示我们编写了一行不遵循 Python 语法规则的代码。它相当于英文语法中的语法错误，例如不以句号结束一个句子，或者忘记将句子的第一个单词大写。例如，在下面的语句行中，由于 `def foo(x, y)` 行末尾缺少冒号 (`:`) 而导致了语法错误。在使用 `if` 语句时，经常会出现类似的语法错误。

```
>>> def foo(x, y)
SyntaxError: invalid syntax
```

另一个常见错误是使用尚未赋值的变量。在下面的示例中，在 `spam` 尚未赋值时就使用了变量 `spam`。请注意，此错误消息以单词 `Traceback` 开头，它可以帮助我们确定发生错误的行和函数。回溯信息之后是错误消息本身，它表示发生了 `NameError`。此消息告诉我们 '`spam`' 未定义。因为在 Python 中定义变量的唯一方法是为其赋值，所以此消息应该帮助我们认识到，在将 `spam` 用于表达式之前，需要为其赋值。

```
>>> 4 + spam * 3
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
NameError: name 'spam' is not defined
```

让我们看看另一个使用保存在 `test.py` 文件（见程序清单 B-1）中的简单 Python 程序的回溯信息的示例。

程序清单 B-1 一个简单的 Python 回溯信息

```
1      def foo():
2          a = 1 + 2
3          b = a + spam
4          return b
5
6      def main():
7          print('hello')
8          x = foo()
9          print(x)
10
11     main()
```

运行上述程序时，会产生以下输出结果：

```
hello
Traceback (most recent call last):
  File "test.py", line 11, in <module>
    main()
  File "test.py", line 8, in main
    x = foo()
  File "test.py", line 3, in foo
    b = a + spam
NameError: name 'spam' is not defined
```

回溯信息的第一行指向 `test.py` 文件的第 11 行代码，其中调用 `main()`。文件的下一行指向第 8 行代码，其中调用 `foo()`。最后一行指向第 3 行代码，在这里我们找到表达式 `b = a+spam`。Python 再次告诉我们没有定义 `spam`，它给出了包含错误的确切行。

当处理混合字符串和数字数据类型时，我们可以尝试将两个不兼容的内容“累加”在一起。下一个示例显示了一个试图累加字符串和整数的表达式。Python 不知道应该将“2”转换为整数并累加两个整数，还是将 2 转换为字符串并拼接两个字符串。因此，它将向我们发送以下错误消息：

```
>>> '2' + 2
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    '2' + 2
TypeError: can only concatenate str (not "int") to str
```

以下是另一个示例情况，Python 不知道如何对加法运算符求值：

```
>>> def bar():
    return 2 * 5

>>> a = 3 + bar
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    a = 3 + bar
TypeError: unsupported operand type(s) for +: 'int' and 'function'
```

在本例中，语句 `a = 3+bar` 缺少名为 `bar` 的函数后面的括号。尽管 `bar` 不需要任何参数，但如果不使用“调用运算符”`()`，Python 就不知道应该调用函数。

下面是对非函数对象使用调用运算符的示例：

```
>>> b = 1
>>> c = 2 + b()
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    c = 2 + b()
TypeError: 'int' object is not callable
```

下一个错误很容易识别，但经常会发生：

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    10 * (1/0)
ZeroDivisionError: division by zero
```

在最后的示例中，当尝试遍历非序列的内容时，会得到以下错误消息。在本例中，`d` 是一个整数，而不是字符串、列表、元组、字典或者文件。

```
>>> d = 3
>>> for i in d:
    print(i)

Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    for i in d:
TypeError: 'int' object is not iterable
```

turtle 参考

有关 turtle 模块的终极指南，请参考 <http://docs.python.org/3.8/library/turtle.html>。

C.1 基本移动和绘制

forward(distance): 沿当前方向向前移动 distance 个单位。

backward(distance): 沿当前方向向后移动 distance 个单位。

right(angle): 把 turtle 向右旋转 angle 度。

left(angle): 把 turtle 向左旋转 angle 度。

goto(pos, y=None): 把 turtle 移动到一个绝对位置。如果 turtle 尾巴处于放下状态，那么绘制一条直线。turtle 的朝向保持不变。可以使用一个数值对或者表示 (x,y) 坐标的元组指定位置。

setx(x): 把 turtle 移动到一个绝对位置 (x,y)，其中 x 作为参数指定，y 保持不变。

sety(y): 把 turtle 移动到一个绝对位置 (x,y)，其中 y 作为参数指定，x 保持不变。

setheading(direction): 设置 turtle 指向的方向。常用的方向角度包含 0= 向右、90= 向上、180= 向左、270= 向下。

home(): 把 turtle 移动到原点位置 (0,0)，并设置其朝向为初始方向。

circle(radius, angles=None, sides=None): 绘制一个圆或者一条圆弧，并根据角度 angle 改变 turtle 的方向。中心在 turtle 左边（考虑到它的朝向）的 radius 个单位。sides 参数指定用于绘制圆的多边形的边数。如果省略，则自动确定边数。

dot(size=None, *color): 在当前位置绘制一个大小为 size、颜色为 color 的圆点，可以使用颜色字符串或者颜色元组指定 color。

```
dot(30)
dot(40, 'blue')
```

undo(): 取消 turtle 的上一次操作。

write(text, move=False, align='left', font=('Arial', 8, 'normal')): 在 turtle 的当前位置输出一些文本。如果 move 为 False，turtle 的画笔不会移动。align 的取值范围为 'left'、'right' 或者 'center'。font 参数允许用户选择消息的字体和大小。

C.2 turtle 状态

position(): 返回 turtle 的当前位置，结果为元组 (x,y)。

towards(pos, y=None): 返回 turtle 的当前朝向与作为参数的位置 pos 之间的角度。

```
towards(number, number)    # 使用 x 和 y 作为位置
towards((number, number))  # 使用元组作为位置
towards(otherTurtle)        # 另一个 turtle
```

xcor(): 返回 turtle 的当前 x 坐标。

ycor(): 返回 turtle 的当前 y 坐标。

heading(): 返回 turtle 的朝向。

distance(pos, y=None): 返回 turtle 的当前位置与作为参数的位置 pos 之间的距离。

```
distance(number, number)    # 使用 x 和 y 作为位置
distance((number, number))  # 使用元组作为位置
distance(otherTurtle)       # 另一个 turtle
```

C.3 绘制状态

down(): 放下 turtle 的尾巴。

up(): 抬起 turtle 的尾巴。

width(width=None): 更改绘制的线宽。如果没有传递参数, 该方法返回当前的线宽。

C.4 填充

begin_fill(): 在绘制需要填充的形状之前调用 begin_fill。

end_fill(): 在绘制需要填充的形状之后调用 end_fill。例如, 使用以下代码片段可以绘制一个填充正方形。

```
turtle.begin_fill()
for i in range(4):
    turtle.forward(100)
    turtle.right(90)
turtle.end_fill()
```

C.5 其他绘制控制

reset(): 从窗口删除 turtle 的所有绘图。将 turtle 重新居中, 并将其变量重置为默认值。

clear(): 删除 turtle 的所有绘图, 但将 turtle 保留在当前位置。

C.6 控制形状和外观

默认情况下, turtle 不限于默认的三角形形状, 而是可以显示为 turtle 形状, 或者可以使用 turtle 绘制的任何其他形状。此外, 还可以使用 gif 文件为 turtle 提供形状。一旦在程序中添加了一个形状, 任何 turtle 都可以通过名称使用该形状。

addshape(name, shape=None): 添加一个新形状到 TurtleScreen 的形状列表中。在使用一个形状之前, 必须先注册该形状。通过使用屏幕对象或者 turtle 模块名称调用此方法。各参数的含义如下:

- 如果 name 是一个 gif 文件的名称, shape 为 None, 那么注册该图像文件。
- 如果 name 是一个字符串, shape 是两个坐标位置的元组, 那么使用这两个位置创建一个矩形作为 turtle 的形状。
- 如果 name 是一个字符串, shape 是一个形状对象, 那么注册该形状对象。

resizemode(rmode=None): 将“调整大小模式”设置为以下值之一: auto、user 或者 noresize。auto (自动) 根据 pensize 自动调整 turtle 的外观。user (用户) 根据 stretchfactor 参数和轮廓宽度的值调整 turtle 的外观。这两种都可以通过 turtlesize.

noresize 强迫 turtle 保持原来的大小。如果未传递任何参数，则该方法返回当前模式。

turtlesize(stretch_wid=None, stretch_len=None, outline=None)：如果 **resizemode** 为 **user**，那么 turtle 将根据参数值拉伸其宽度和长度。**outline** 参数指定形状轮廓的宽度。

getshapes()：返回 TurtleScreen 形状列表中的所有形状。默认的形状列表为 ['arrow', 'blank', 'circle', 'classic', 'square', 'triangle', 'turtle']。

begin_poly()：开始记录多边形的顶点。

end_poly()：停止记录多边形的顶点。

get_poly()：返回最后记录的多边形。多边形是可以传递给 **addshape** 的对象。

shape(name=None)：设置 turtle 形状。如果参数 **name** 为 **None**，则返回 turtle 形状。创建自定义形状的方法参见以下示例：

```

turtle.begin_poly()
for i in range(4):
    turtle.forward(10)
    turtle.right(90)
turtle.end_poly()
squareShape = turtle.get_poly()
turtle.addshape('squarepants', squareShape)
turtle.shape('squarepants')

```

hideturtle()：设置 turtle 不可见。

showturtle()：设置 turtle 可见。

C.7 度量设置

degrees(fullcircle=360.0)：将角度的度量单位更改为度。对于某些应用，一个完整的圆可以有不同的度数。例如，我们可能希望使圆具有 100 度，以便于将百分比映射到圆的各个部分。我们可以使用 **fullcircle** 参数修改此参数。

radians()：将角度的度量单位更改为弧度。

C.8 绘制速度

speed(speed=None)：如果没有指定参数 **speed**，那么返回 turtle 的当前速度；否则，设置速度为 0 到 10 范围之间的值，其中 0 表示静止，1 表示最慢。速度随数值而增加，最大速度为 10。

tracer(n=None, delay=None)：在设置 turtle 动画时，每 **n** 次更新执行一次。第二个参数指定延时值，单位为毫秒。

delay(delay=None)：设置或者返回绘制延时值，单位为毫秒。

update()：更新屏幕中的所有图形。适用于当 **tracer** 设置为 0 的情况。

C.9 颜色

bgcolor(*args)：设置或者返回 TurtleScreen 的背景颜色。可以通过以下几种方式设置颜色：

- **bgcolor('colorname')**：Python 可以识别大多数合理的颜色名称。

- `bgcolor(r, g, b)`: `r`、`g` 和 `b` 是位于 0.0 到 1.0 之间的浮点值。
- `bgcolor((r, g, b))`: `r`、`g` 和 `b` 是位于 0.0 到 1.0 之间的浮点值。

`bgpic(picname=None)`: 将窗口的背景设置为指定文件名的图像。`picname` 可以是字符串、gif 格式的文件名、`nopic` 或者 `None`。如果为 `nopic`，则删除当前背景图像。如果为 `None`，则返回当前背景图像。

`pencolor(*args)`: 设置画笔的颜色，使用与 `bgcolor` 相同的选项。

```
pencolor('colorname')
pencolor(r, g, b)
pencolor((r, g, b))
```

有关详细说明信息，请参见 `color`。

`fillcolor(*args)`: 设置当前的填充色，使用与 `bgcolor` 相同的选项。

`color(*args)`: 在没有参数的情况下调用时，返回当前画笔和填充颜色。否则，可以使用此方法在一次调用中设置 `pencolor` 和 `fillcolor`。调用此方法的合法方式包括：

```
color(rp, gp, bp, rf, gf, bf)
color((rp, gp, bp), (rf, gf, bf))
color('pencolorname', 'fillcolorname')
```

第一组数值用于设置画笔颜色；第二组数值用于设置填充颜色。

`colormode(cmode=None)`: 将红 - 绿 - 蓝颜色的规范更改为介于 0 和 1 之间的浮点值或者介于 0 和 255 之间的整数值。调用 `colormode(255)` 将更改为整数模式。默认值为 1.0。

```
colormode(255) # 范围为 0 ~ 255
color(200, 149, 58)
colormode(1.0) # 范围为 0.0 ~ 1.0
color(.75, .69, .02)
```

C.10 事件

`onclick(function, button=1, add=None)`: 将一个函数绑定到鼠标单击事件，以便每当用户按下鼠标按钮时调用 `function`。请注意，`function` 必须接收两个参数，这两个参数将在单击按钮时给出鼠标的位置。`button` 是鼠标按钮，1 是左键。如果 `add` 为 `True`，将添加一个新的回调函数；否则，函数将替换以前的回调函数。如果 `None` 作为函数参数传递，则取消回调。

`onkey(function, key)`: 将函数绑定到按键事件。`function` 必须定义为不带参数的函数。`key` 是表示键盘上按键的字符串。如果 `None` 作为函数参数传递，则取消回调。

`ontimer(function, time=0)`: 将函数绑定到计时器事件。经过 `time` 毫秒后，`function` 被调用一次。`function` 必须定义为不带参数的函数。

`listen(x=None, y=None)`: 把焦点放在当前 `turtle` 的画布上，这样它就可以监听关键事件。在调用此函数之前，将无法识别按键事件。此 `listen` 方法被定义为接受两个伪参数，以便可以将 `listen` 绑定到鼠标单击事件。

`mainloop()`: 启动事件循环。此句必须是 `turtle` 图形程序中的最后一个语句。

C.11 其他设置

`turtles()`: 返回屏幕上所有 `turtle` 的列表。

clearscreen() : 从屏幕上删除所有图形和所有 turtle。屏幕被设置为白色背景, 没有事件绑定, 并且跟踪处于打开状态。

resetscreen() : 将屏幕上的所有 turtle 重置为其初始位置。

screensize(canvwidth=None, canvheight=None, bg=None) : 将绘图画布大小调整为 canvwidth 和 canvheight。不调整窗口的大小; 只需为不在原始绘图区域中的查看区域添加滚动条。如果未给出参数, 则返回当前宽度和高度。

bye() : 关闭 turtle 图形窗口。

exitonclick() : 在窗口内等待鼠标单击, 然后关闭并退出。

setworldcoordinates(llx, lly, urx, ury) : 修改绘图画布的世界坐标。这将自动在 x 方向的 llx (左下 x) 和 urx (右上 x) 之间以及 y 方向的 lly (左下 y) 和 ury (右上 y) 之间缩放画布。

部分“动手实践”参考答案

第 1 章

1.1

```
>>> 8 + 9 + 10
27
```

1.3

```
>>> 365 * 24 * 60 * 60 #days * hours * minutes * seconds
31536000
```

1.6

```
>>> (17 * (17 + 1)) // 2 # 假设班级中有 18 个同学
153
```

1.9

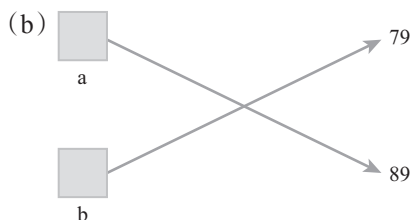
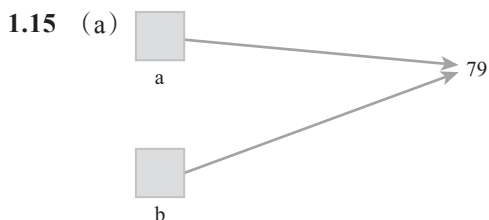
```
>>> 2 ** 120
1329227995784915872903807060280344576
```

1.10

```
>>> (18 + 19 + 19 + 20 + 25) / 5
20.2
```

1.13

```
>>> 2900000 * 5.878e12
1.70462e+19
```



1.18 a 是 20; b 是 15

1.21

```
>>> ole = turtle.Turtle()
>>> ole.right(45)
>>> ole.forward(50)
```

1.24

```
def drawRectangle(myTurtle, width, height):
    myTurtle.forward(width)
    myTurtle.left(90)
    myTurtle.forward(height)
    myTurtle.left(90)
    myTurtle.forward(width)
    myTurtle.left(90)
    myTurtle.forward(height)
    myTurtle.left(90)
```

1.27 从图 1-12 开始, 存在一个新的名称 drawRectangle, 该名称指向一个函数。该函数包含对应于 myTurtle、width 和 height 的参数名称。

1.29

```
range(-10, 11, 1)
```

1.32

```
def drawSpiral(myTurtle, maxSide):
    for sideLength in range(1, maxSide + 1, 5):
        myTurtle.forward(sideLength)
        myTurtle.right(75)
```

1.34

```
def drawSpiral(turtle1, turtle2, maxSide):
    for sideLength in range(1, maxSide + 1, 5):
        turtle1.forward(sideLength)
        turtle2.forward(sideLength)
        turtle1.right(90)
        turtle2.left(90)
```

```

1.36 def drawSquare(myTurtle, sideLength):
    myTurtle.forward(sideLength)
    myTurtle.left(90)
    myTurtle.forward(sideLength)
    myTurtle.left(90)
    myTurtle.forward(sideLength)
    myTurtle.left(90)
    myTurtle.forward(sideLength)
    myTurtle.left(90)

    def drawNestedSquares(myTurtle):
        for i in range(200, 200 - 5 * 10, -5):
            drawSquare(myTurtle, i)

1.39 def plotSimple(myTurtle):
    myTurtle.up()
    startX = -100
    endX = 101
    myTurtle.goto(startX, startX/2 + 3)
    myTurtle.down()
    for x in range(startX, endX, 1):
        y = x / 2.0 + 3
        myTurtle.goto(x, y)

1.40 def plotCircle(myTurtle, radius):
    circumference = 2 * 3.1415 * radius
    sideLength = circumference / 360
    myTurtle.up()
    myTurtle.left(90)
    myTurtle.forward(radius)
    myTurtle.right(90)
    myTurtle.down()
    drawPolygon(myTurtle, sideLength, 360) #from Listing 1.5
    myTurtle.up()
    myTurtle.right(90)
    myTurtle.forward(radius)
    myTurtle.left(90)
    myTurtle.down()

```

第 2 章

2.3 运行动手实践 2.2 中的语句 `help(turtle)`，将产生以下错误信息：

```
NameError: name 'turtle' is not defined
```

但是，在导入 `turtle` 模块之后：

```
import turtle
```

重新运行动手实践 2.2 中的语句，将输出有关 `turtle` 模块的帮助信息。

2.5 在导入 `random` 模块后，键入命令 `help('random')`，我们发现 `randrange` 返回一个从下边界到上边界的范围中的随机整数，但不包含上边界。`randint` 函数则包含上边界。

```

2.7 import math
def archimedes(sides, radius):
    innerAngleB = 360.0 / sides
    halfAngleA = innerAngleB / 2
    oneHalfSideS = radius * math.sin(math.radians(halfAngleA))
    sideS = oneHalfSideS * 2
    polygonCircumference = sides * sideS

```

```
pi = polygonCircumference / (2 * radius)
return pi
```

与圆的大小无关。根据误差水平，大约需要 1.4 亿到 1.5 亿条边才能接近圆周率。

2.10 >>> acc = 0

```
>>> for x in range(1, 200, 2):
    acc = acc + x
```

```
>>> average = acc / 100
>>> average
100.0
```

2.13 >>> older = 1

```
>>> oldest = 1
>>> for next in range(3, 11):
    newFib = older + oldest
    oldest = older
    older = newFib
```

```
>>> print(newFib)
55
```

2.17 def leibniz2(terms):

```
    acc = 0
    num = 4
    den = 1
    multiplier = 1
    for aTerm in range(terms):
        nextTerm = num / den * multiplier
        multiplier = multiplier * -1
        acc = acc + nextTerm
        den = den + 2
    return acc
```

2.19 使用参数值 100 000 比较各个函数，结果表明，archimedes 函数最精确，其次是 wallis 函数、最后是 leibniz 函数。

2.22 False

2.25 False

2.27 count >= 1 and count <= 10

2.30 if score >= 90:

```
    gradePoint = 4
elif score >= 80:
    gradePoint = 3
elif score >= 70:
    gradePoint = 2
elif score >= 60:
    gradePoint = 1
else:
    gradePoint = 0
```

2.33 def costOfOranges(pounds):

```
    costFruit = pounds * 0.32
    shipping = 7.50
    if pounds > 100:
        shipping = shipping - 1.50
    totalCost = costFruit + shipping
    return totalCost
```

2.35 def paycheck(rate, hours):

```
    if hours <= 40:
        pay = rate * hours
    else:
        pay = rate * 40 + (1.5 * rate * (hours - 40))
    return pay
```

```
2.38 def isInCircle(x, y, radius):
    distance = math.sqrt(x**2 + y**2)

    if distance <= radius:
        return True
    else:
        return False

2.40 import random
import math
import turtle

def showMontePi(numDarts):
    wn = turtle.Screen()
    drawingT = turtle.Turtle()
    wn.setworldcoordinates(0, 0, 1, 1)
    drawingT.up()
    drawingT.goto(0, 0)
    drawingT.down()
    drawingT.goto(1, 0)
    drawingT.up()
    drawingT.goto(0, 0)
    drawingT.down()
    drawingT.goto(0, 1)
    circle = 0
    drawingT.up()
    for i in range(numDarts):
        x = random.random()
        y = random.random()
        distance = math.sqrt(x**2 + y**2)
        drawingT.goto(x, y)
        if distance <= 1:
            circle = circle + 1
            drawingT.color("blue")
        else:
            drawingT.color("red")
        drawingT.dot()

    pi = circle / numDarts * 4
    wn.exitonclick()
    return pi
```

第 3 章

```
3.2 fullName[:x], 其中 x 是英文名后面空格的索引。

3.5 print(len(fullName[:x]))

3.7 for i in range(len(name) + 1):
    print(name[:i])

3.10 >>> river = "mississippi"
>>> river.find("p")
8
>>> river.index("p")
8

3.13 def toInt(ch):
    if ord(ch) > ord('9') or ord(ch) < ord('0'):
        print('error:', ch, 'is not a digit')
        return -1
    else:
        return ord(ch) - ord('0')

3.16 def letterGrade(score):
    if score >= 90:
```

```

        letter = 'A'
    elif score >= 80:
        letter = 'B'
    elif score >= 70:
        letter = 'C'
    elif score >= 60:
        letter = 'D'
    else:
        letter = 'F'
    return letter

```

3.18 此解决方案更为通用，因为它将删除 `removeString` 中的任何字符。因此，若要去掉空格，请使用 " " 作为 `removeString` 参数调用此方法。

```

def removeMatches(myString, removeString):
    newStr = ""
    for ch in myString:
        if ch not in removeString:
            newStr = newStr + ch
    return newStr

```

3.22 `def substitutionEncryptNoSpaces(plainText, key):`
`alphabet= "abcdefghijklmnopqrstuvwxyz"`
`plainText = plainText.lower()`
`plainText = plainText.replace(" ", "")`
`cipherText=""`
`for ch in plainText:`
 `idx = alphabet.find(ch)`
 `cipherText = cipherText + key[idx]`
`return cipherText`

3.25 `def substitutionEncrypt(plainText, password):`
`key = genKeyFromPass(password)`
`alphabet = "abcdefghijklmnopqrstuvwxyz "`
`plainText = plainText.lower()`
`cipherText = ""`
`for ch in plainText:`
 `idx = letterToIndex(ch)`
 `cipherText = cipherText + key[idx]`
`return cipherText`

3.27 `def rotEncrypt(message, rotNum):`
`alphabet = "abcdefghijklmnopqrstuvwxyz"`
`message = message.replace(" ", "")`
`message = message.lower()`
`result = ""`
`for ch in message:`
 `idx = alphabet.find(ch)`
 `idx = (idx + rotNum) % len(alphabet)`
`result = result + alphabet[idx]`
`return result`

```

def rotDecrypt(cipherText, rotNum):
    return rotEncrypt(cipherText, -rotNum)

```

3.28 `def undoVig(keyLetter, cipherText):`
`keyIndex = letterToIndex(keyLetter)`
`ctIndex = letterToIndex(cipherText)`
`newIdx = (ctIndex - keyIndex) % 26`
`return indexToLetter(newIdx)`

第 4 章

4.2 `>>> myList = [7, 9, "a", "cat", False]`
`>>> myList`

```

[7, 9, 'a', 'cat', False]
>>> myList.append(3.14)                    # (a)
>>> myList.append(7)
>>> myList
[7, 9, 'a', 'cat', False, 3.14, 7]
>>> myList.insert(3, "dog")                 # (b)
>>> myList
[7, 9, 'a', 'dog', 'cat', False, 3.14, 7]
>>> myList.index("cat")                     # (c)
4
>>> myList.count(7)                         # (d)
2
>>> myList.remove(7)                        # (e)
>>> myList
[9, 'a', 'dog', 'cat', False, 3.14, 7]
>>> myList.pop(myList.index("dog"))         # (f)
'dog'
>>> myList
[9, 'a', 'cat', False, 3.14, 7]

```

```

4.5 def wordCount(sentence):
    wordList = sentence.split()
    return len(wordList)

```

```

4.7 import random
def shuffle(aList):
    copyList = aList[:]
    newList = []
    for i in range(len(copyList)):
        item = copyList.pop(random.randrange(len(copyList)))
        newList.append(item)
    return newList

```

```

4.10 >>> myList = [[]]*3
>>> myList
[ [], [], [] ]

```

该语句创建一个列表，包含一个空列表的三个副本。

```

4.13 >>> myList = []
>>> for i in range(3):
    myList.append([])

```

```

>>> myList
[[], [], []]
>>> myList[1].append(2)
>>> myList
[[], [2], []]

```

```

4.15 def getMin(aList):
    minSoFar = aList[0]
    for item in aList[1:]:
        if item < minSoFar:
            minSoFar = item
    return minSoFar

```

```

4.18 def mean(aList):
    sumOfItems = 0
    for item in aList:
        sumOfItems = sumOfItems + item
    mean = sumOfItems / len(aList)
    return mean

```

```

4.24 def makeDictionary(list1, list2):
    newDict = {}
    for i in range(len(list1)):
        newDict[list1[i]] = list2[i]
    return newDict
>>> scoreDict = makeDictionary(names, scores)

```

```

4.25 >>> scoreDict.get('barb')
13
4.27 >>> scoresList = list(scoreDict.values())
>>> scoresList.sort()
>>> scoresList
[10, 12, 13, 18, 23]
4.31 >>> scoreDict
{'joe': 10, 'barb': 13, 'sue': 18, 'sally': 13, 'john': 19}
>>> keys = scoreDict.keys()
>>> keys = list(keys)
>>> keys
['joe', 'barb', 'sue', 'sally', 'john']
>>> keys.sort()
>>> keys
['barb', 'joe', 'john', 'sally', 'sue']
>>> for aName in keys:
    print(aName, scoreDict[aName])

barb 13
joe 10
john 19
sally 13
sue 18
4.33 def mode(aList):
    maxValue = max(aList)
    countList = [0] * (maxValue + 1)
    for item in aList:
        countList[item] = countList[item] + 1
    maxCount = max(countList)
    modelist = []
    for i in range(len(countList)):
        if countList[i] == maxCount:
            modelist.append(i)
    return modelist
4.35 def averages(scoreList):
    dataDict = {}
    for item in scoreList:
        name = item[0]
        score = item[1]
        if name in dataDict:
            dataDict[name].append(score)
        else:
            dataDict[name] = [score]
    for aKey in dataDict:
        print(aKey, sum(dataDict[aKey]) / len(dataDict[aKey]))
4.38 def frequencyTable(aList):
    countDict = {}
    for item in aList:
        if item in countDict:
            countDict[item] = countDict[item] + 1
        else:
            countDict[item] = 1
    items = countDict.items()
    items = list(items)
    items.sort()
    print("ITEM", "FREQUENCY")
    for item in items:
        print(item[0], "      ", item[1])
4.42 >>> import turtle
>>> import statistics
>>> def frequencyChart(aList):

```

```

countDict = {}                                # 生成计数器字典

for item in aList:
    if item in countDict:
        countDict[item] = countDict[item] + 1
    else:
        countDict[item] = 1

itemList = list(countDict.keys())              #g# 获取 dict 中的键列表
minItem = 0
maxItem = len(itemList) - 1

countList = countDict.values()                #g# 获取计数器列表
maxCount = max(countList)

wn = turtle.Screen()
chartT = turtle.Turtle()
wn.setworldcoordinates(-1, -1, maxItem + 4, maxCount + 1)
chartT.hideturtle()

chartT.up()                                    #d# 绘制 x 轴基线
chartT.goto(0, 0)
chartT.down()
chartT.goto(maxItem + 1, 0)
chartT.up()

chartT.goto(-1, 0)                            #f# 显示 y 轴的最小值和最大值
chartT.write("0", font=("Helvetica", 16, "bold"))
chartT.goto(-1, maxCount)
chartT.write(str(maxCount), font=("Helvetica", 16, "bold"))

for index in range(len(itemList)):
    chartT.goto(index, -1)                    ## 标记项的值
    chartT.write(str(itemList[index]),
                  font=("Helvetica", 16, "bold"))
    chartT.goto(index, 0)                    ## 绘制高为 count 的直线
    chartT.down()
    chartT.goto(index, countDict[itemList[index]])
    chartT.up()

chartT.goto(len(itemList), -1)                ## 绘制均值
chartT.write(str(statistics.mean(countDict.values()))
             + "(mean)", font=("Helvetica", 16, "bold"))
chartT.goto(len(itemList), 0)
chartT.down()
chartT.goto(len(itemList),
             statistics.mean(countDict.values()))
chartT.up()
wn.exitonclick()

```

第 5 章

- 5.1** >>> with open("rainfall.txt", "r") as rainFile:
 with open("rainfallFmt.txt", "w") as outFile:
 for aLine in rainFile:
 values = aLine.split()
 city = values[0]
 inches = float(values[1])
 nChars = outFile.write(\n
 "{0:>25s} {1:5.1f}\n".format(city, inches))
- 5.3** >>> with open("rainfall.txt", "r") as inFile:
 line1 = inFile.readline()


```
line2 = inFile.readline()
restOfFile = inFile.readlines()
```

```
>>> restOfFile
['Algona 30.69\n', 'Allison 33.64\n', 'Alton 27.43\n', 'AmesW
34.07\n', 'AmesSE 33.95\n', 'Anamosa 35.33\n', 'Ankeny 33.38\n',
'Atlantic 34.77\n', 'Audubon 33.41\n', 'Beaconsfield 35.27\n',
'Bedford 36.35\n', 'BellePlaine 35.81\n', 'Bellevue 34.35\n',
'Blockton 36.28\n', 'Bloomfield 38.02\n', 'Boone 36.30\n', 'Brighton
33.59\n', 'Britt 31.54\n', 'Buckeye 33.66\n', 'BurlingtonKBUR
37.94\n', 'Burlington 36.94\n', 'Carroll 33.33\n', 'Cascade 33.48\n']
```

注: `readlines` 方法返回文件中从第 3 行到末尾的内容。

```
5.6 >>> with open("rainfall.txt", "r") as inFile:
    lineCount = 0
    wordCount = 0
    charCount = 0
    for aLine in inFile:
        lineCount = lineCount + 1
        wordList = aLine.split()
        wordCount = wordCount + len(wordList)
        charCount = charCount + len(aLine[0:-1])
    print("Lines Words Characters")
    print("{0:>5} {1:>5} {2:>10}".format(lineCount, \
        wordCount, charCount))
```

```
5.9 import csv
def makeMagListByDate():
    magDict = {}
    with open("earthquakes.csv", "r") as inFile:
        csvReader = csv.reader(inFile)
        colNames = next(csvReader) #skip title line
        for aLine in csvReader:
            mag = aLine[4]
            dateTime = aLine[0]
            dateTimeValues = dateTime.split("T")
            date = dateTimeValues[0]
            if date in magDict:
                magDict[date].append(mag)
            else:
                magDict[date] = [mag]
            dataList = []
            for aDate in magDict:
                dataList.append([aDate] + magDict[aDate])
    return dataList
```

```
5.12 >>> i = 10
>>> while i > -1:
    print("Hello", i)
    i = i - 1
```

```
5.15 def getProperty(data, key):
    valueList = []
    earthquakes = data.get('features')
    for i in range(len(earthquakes)):
        features = earthquakes[i]
        props = features.get('properties')
        value = props.get(key)
        valueList.append(value)
    return valueList
```

```
5.17 >>> feltShortList = []
>>> earthquakes = eData.get('features')
>>> for i in range(len(earthquakes)):
    features = earthquakes[i]
    props = features.get('properties')
```

```

    felt = props.get('felt')
    if (felt != None):
        feltShortList.append(felt)

```

第 6 章

6.2 def drawRectangle():

```

    rectX = 100
    rectY = 100
    rectWidth = 100
    rectLength = 125
    whitePixel = Pixel(255, 255, 255)
    im = EmptyImage(300, 300)
    # 绘制顶部和底部直线
    for i in range(rectX, rectX + rectWidth):
        im.setPixel(i, rectY, whitePixel)
        im.setPixel(i, rectY + rectLength, whitePixel)
    # 绘制左边和右边直线
    for i in range(rectY, rectY + rectLength):
        im.setPixel(rectX, i, whitePixel)
        im.setPixel(rectX + rectWidth, i, whitePixel)
    return im

>>> imWin = ImageWin("Drawing a Rectangle", 300, 300)
>>> rectWin = drawRectangle()
>>> rectWin.draw(imWin)

```

6.6 每种颜色强度都存储在一个字节（8 位二进制）中。可以存储在 8 位二进制中的最大值为 255。每种颜色强度范围从 0 到 255，即每种强度有 256 个值。

6.7 def clearRed(oldPixel):

```

    newPixel = Pixel(0, oldPixel.getGreen(), oldPixel.getBlue())
    return newPixel

```

6.11 def blackWhite(oldPixel):

```

    red = oldPixel.getRed()
    green = oldPixel.getGreen()
    blue = oldPixel.getBlue()
    gray = (red + green + blue) / 3
    if gray <= 128:
        newPixel = Pixel(0, 0, 0)
    else:
        newPixel = Pixel(255, 255, 255)
    return newPixel

```

6.17 def sepia(oldPixel):

```

    red = oldPixel.getRed()
    green = oldPixel.getGreen()
    blue = oldPixel.getBlue()
    newRed = int(red*0.393 + green*0.769 + blue*0.189)
    newGreen = int(red*0.349 + green*0.686 + blue*0.168)
    newBlue = int(red*0.272 + green*0.534 + blue*0.131)
    if newRed > 255:
        newRed = 255
    if newGreen > 255:
        newGreen = 255
    if newBlue > 255:
        newBlue = 255
    newPixel = Pixel(newRed, newGreen, newBlue)
    return newPixel

```

6.22 def fourTimes(oldImage):

```

    oldW = oldImage.getWidth()
    oldH = oldImage.getHeight()
    newIm = EmptyImage(oldW * 4, oldH * 4)

```

```

for row in range(newIm.getHeight()):
    for col in range(newIm.getWidth()):
        origCol = col // 4
        origRow = row // 4
        oldPixel = oldImage.getPixel(origCol, origRow)
        newIm.setPixel(col, row, oldPixel)
return newIm

```

6.24 `def halfSize(oldImage):`

```

newWidth = oldImage.getWidth() // 2
newHeight = oldImage.getHeight() // 2
newIm = EmptyImage(newWidth, newHeight)
for row in range(newHeight):
    for col in range(newWidth):
        # 获取邻居平均值
        n1Pixel = oldImage.getPixel(2 * col, 2 * row)
        n2Pixel = oldImage.getPixel(2 * col + 1, 2 * row)
        n3Pixel = oldImage.getPixel(2 * col, 2 * row + 1)
        n4Pixel = oldImage.getPixel(2 * col + 1, 2 * row + 1)
        red = (n1Pixel.getRed() + n2Pixel.getRed() \
              + n3Pixel.getRed() + n4Pixel.getRed()) // 4
        green = (n1Pixel.getGreen() + n2Pixel.getGreen() \
              + n3Pixel.getGreen() + n4Pixel.getGreen()) // 4
        blue = (n1Pixel.getBlue() + n2Pixel.getBlue() \
              + n3Pixel.getBlue() + n4Pixel.getBlue()) // 4
        newPixel = Pixel(red, green, blue)
        newIm.setPixel(col, row, newPixel)
return newIm

```

6.26 `import statistics`

```

def reduceNoise(oldImage):
    height = oldImage.getHeight()
    width = oldImage.getWidth()
    newIm = EmptyImage(width, height)
    for row in range(height - 1):
        for col in range(width - 1):
            n1 = oldImage.getPixel(col, row)
            n2 = oldImage.getPixel(col, row + 1)
            n3 = oldImage.getPixel(col + 1, row)
            n4 = oldImage.getPixel(col + 1, row + 1)
            red = [n1.getRed(), n2.getRed(), n3.getRed(), \
                  n4.getRed()]
            green = [n1.getGreen(), n2.getGreen(), \
                  n3.getGreen(), n4.getGreen()]
            blue = [n1.getBlue(), n2.getBlue(), n3.getBlue(), \
                  n4.getBlue()]
            medRed = int(statistics.median(red))
            medGreen = int(statistics.median(green))
            medBlue = int(statistics.median(blue))
            newPixel = Pixel(medRed, medGreen, medBlue)
            newIm.setPixel(col, row, newPixel)
        # 将最后像素点设置为 unchanged
    for row in range(height - 1):
        oldPixel = oldImage.getPixel(width - 1, row)
        newIm.setPixel(width - 1, row, oldPixel)
    for col in range(width - 1):
        oldPixel = oldImage.getPixel(col, height - 1)
        newIm.setPixel(col, height - 1, oldPixel)
    return newIm

```

6.28 `def completeFlip(oldImage):`

```

oldW = oldImage.getWidth()
oldH = oldImage.getHeight()
newIm = EmptyImage(oldW, oldH)

```

```

        for row in range(oldH):
            for col in range(oldW):
                oldPixel = oldImage.getPixel(oldW - 1 - col, \
                                                oldH - 1 - row)
                newIm.setPixel(col, row, oldPixel)
    return newIm
6.29 def verticalMirror(oldImage):
    oldW = oldImage.getWidth()
    oldH = oldImage.getHeight()
    newIm = EmptyImage(oldW, oldH)
    if oldW % 2 == 0:
        halfway = oldW // 2
    else:
        halfway = oldW // 2 + 1
    maxP = oldW - 1
    for row in range(oldH):
        for col in range(halfway):
            oldPixel = oldImage.getPixel(maxP - col, row)
            newIm.setPixel(maxP - col, row, oldPixel)
            newIm.setPixel(col, row, oldPixel)
    return newIm
6.32 def rotate90(oldImage):
    oldW = oldImage.getWidth()
    oldH = oldImage.getHeight()
    newIm = EmptyImage(oldH, oldW)
    rightCol = oldH - 1
    for row in range(oldH):
        for col in range(oldW):
            oldPixel = oldImage.getPixel(col, row)
            newIm.setPixel(rightCol, col, oldPixel)
            rightCol = rightCol - 1
    return newIm
6.35 较小的数字产生更多的边；较大的数字产生更少的边。实际结果取决于原始图像。

```

第 7 章

```

7.1 def countTo50():
    counter = 0
    while counter <= 50:
        print(counter)
        counter = counter + 5
7.3 def scoreAverage():
    sum = 0
    howMany = 0
    score = input("Please enter a score ('stop' to exit) ")
    while score != 'stop':
        sum = sum + int(score)
        howMany = howMany + 1
        score = input("Please enter a score ('stop' to exit) ")
    average = sum / howMany
    return average
7.7 def createClusters(k, centroids, dataDict, repeats):
    prevClusters = []
    for i in range(k):
        prevClusters.append([])
    aPass = 0
    doAgain = True

    while doAgain:
        print( "****PASS", aPass + 1, "****")
        clusters = []

```

```

for i in range(k):
    clusters.append([])

for aKey in dataDict:
    distances = []
    for clusterIndex in range(k):
        dist = euclidD(dataDict[aKey],
                        centroids[clusterIndex])
        distances.append(dist)
    minDist = min(distances)
    index = distances.index(minDist)

    clusters[index].append(aKey)

    # 将新的聚类与上一个聚类比较
    if clusters != prevClusters:
        prevClusters = clusters
    else:
        doAgain = False

dimensions = len(dataDict[1])

for clusterIndex in range(k):
    sums = [0] * dimensions
    for aKey in clusters[clusterIndex]:
        dataPoints = dataDict[aKey]
        for ind in range(len(dataPoints)):
            sums[ind] = sums[ind] + dataPoints[ind]

    for ind in range(len(sums)):
        clusterLen = len(clusters[clusterIndex])
        if clusterLen != 0:
            sums[ind] = sums[ind] / clusterLen

    centroids[clusterIndex] = sums

for c in clusters:
    print("CLUSTER")
    for key in c:
        print(dataDict[key], end = " ")
    print()
    aPass = aPass + 1
return clusters

```

第 8 章

```

8.1 def createWordList(dName):
    myList = []
    with open(dName, "r") as myFile:
        for aLine in myFile:
            myList.append(aLine[:-1])
    return myList

8.2 import time
def createWordList():
    start = time.time()
    myList = []
    with open('wordlist.txt', 'r') as myFile:
        for line in myFile:
            myList.append(line[:-1])
    end = time.time()
    print('create time', end - start)

```

```

def createWordDict():
    start = time.time()
    myDict = {}
    with open('wordlist.txt', 'r') as myFile:
        for line in myFile:
            myDict[line[:-1]] = True
    end = time.time()
    print('create time', end - start)

```

请注意，创建字典需要更长的时间。

- 8.4** >>> wList = ['the', 'quick', 'brown', 'fox']
>>> " ".join(wList)
'the quick brown fox'
>>> ':'.join(wList)
'the:quick:brown:fox'
>>> ', '.join(wList)
'the,quick,brown,fox'
>>> '--'.join(wList)
'the--quick--brown--fox'
- 8.9** def railBreak(cipherText):
wordDict = createWordDict("wordlist.txt")
cipherLen = len(cipherText)
maxGood = 0
for i in range(1, len(cipherText) // 2):
 if cipherLen % i == 0: # 可能的围栏
 print("trying rails:", i)
 words = railDecrypt(cipherText, i)
 goodCount = 0
 for w in words:
 if w in wordDict:
 goodCount = goodCount + 1
 if goodCount > maxGood:
 maxGood = goodCount
 bestGuess = " ".join(words)
 return bestGuess
- 8.14** def wordPop(text, wordLength):
wordList = []
words = text.split()
for w in words:
 if len(w) == wordLength and w not in wordList:
 wordList.append(w)
 wordList.sort()
 return wordlist
- 8.15** def sortKey(x):
return x[1]
- def endPop(myString):
wordList = myString.split()
popDict = {}
for word in wordList:
 popDict[word[-1]] = popDict.get(word[-1], 0) + 1
popList = list(popDict.items())
popList.sort(key=sortKey, reverse=True)
return popList[0]
- 8.18** re.match('.*ing\$', myWord)
- 8.22** re.match('[a-z][aeiou][aeiou][a-z]\$', myWord)
- 8.23** def getNameSuffix(filename):
g = re.match('(.*?)\.(.*)', filename)
if g:
 return g.group(1), g.group(2)

```

else:
    return None

```

第 9 章

- 9.2** `def drawCenteredSquare(aTurtle, x, y, side):`
`aTurtle.up()`
`aTurtle.goto(x, y)`
`aTurtle.dot()`
`aTurtle.left(90)`
`aTurtle.forward(side / 2)`
`aTurtle.left(90)`
`aTurtle.forward(side / 2)`
`aTurtle.right(180)`
`aTurtle.down()`
`for i in range(4):`
`aTurtle.forward(side)`
`aTurtle.right(90)`
- `def concentricBox(aTurtle, x, y, side):`
`if side >= 1:`
`drawCenteredSquare(aTurtle, x, y, side)`
`concentricBox(aTurtle, x, y, side - 10)`
- `>>> import turtle`
`>>> t = turtle.Turtle()`
`>>> concentricBox(t, 0, 0, 200)`
- 9.4** `def sumList(aList):`
`if aList == []:`
`return 0`
`else:`
`return aList[0] + sumList(aList[1:])`
- 9.7** `def reverseString(aString):`
`if aString == '':`
`return ''`
`else:`
`return reverseString(aString[1:]) + aString[0]`
- 9.10** `def treeLeft(t, trunkLength):`
`if trunkLength < 5:`
`return`
`else:`
`t.forward(trunkLength)`
`t.left(30)`
`treeLeft(t, trunkLength - 15)`
`t.right(60)`
`treeLeft(t, trunkLength - 15)`
`t.left(30)`
`t.backward(trunkLength)`
- 9.12** `import random`
`def tree(t, trunkLength):`
`if trunkLength < 5:`
`return`
`else:`
`t.forward(trunkLength)`
`t.left(30)`
`tree(t, trunkLength - random.randint(5, 25))`
`t.right(60)`
`tree(t, trunkLength - random.randint(5, 25))`
`t.left(30)`
`t.backward(trunkLength)`
- 9.15** `def drawTriangle(t, p1, p2, p3, aColor):`

```

        t.color(aColor)
        t.up()
        t.goto(p1)
        t.down()
        t.goto(p2)
        t.goto(p3)
        t.goto(p1)
def sierpinski(myTurtle, p1, p2, p3, depth, aColor):
    if depth > 0:
        sierpinski(myTurtle, p1, midPoint(p1,p2), midpoint(p1,p3),
                    depth - 1, 'blue')
        sierpinski(myTurtle, p2, midPoint(p2,p3), midpoint(p2,p1),
                    depth - 1, 'green')
        sierpinski(myTurtle, p3, midPoint(p3,p1), midpoint(p3,p2),
                    depth - 1, 'red')
    else:
        drawTriangle(myTurtle, p1, p2, p3, aColor)

```

执行上述函数:

```

>>> import turtle
>>> t = turtle.Turtle()
>>> sierpinski(t, (0,0), (200,200), (200,0), 4, 'black')
>>> t.hideturtle()

```

9.23

```

def recKoch(aT, distance, level):
    if level == 0:
        aT.forward(distance)
    else:
        recKoch(aT, distance / 3, level - 1)
        aT.left(60)
        recKoch(aT, distance / 3, level - 1)
        aT.right(120)
        recKoch(aT, distance / 3, level - 1)
        aT.left(60)
        recKoch(aT, distance / 3, level - 1)

```

```

>>> import turtle
>>> t = turtle.Turtle()
>>> t.up()
>>> t.goto(-100, 100)
>>> t.down()

```

```

>>> recKoch(t, 200, 4)
>>> t.right(120)
>>> recKoch(t, 200, 4)
>>> t.right(120)
>>> recKoch(t, 200, 4)
>>> t.hideturtle()

```

9.25

```

AB
ABA
ABAAB
ABAABABA
ABAABABAABAAB

```

9.28

```

>>> import turtle
>>> t = turtle.Turtle()
>>> axiom = 'F++F++F'
>>> myRules = {'F': 'F-F++F-F'}
>>> instructions = applyProduction(axiom, myRules, 3)
>>> t.up()
>>> t.goto(-100, 100)
>>> t.down()
>>> drawLS(t, instructions, 60, 10)
>>> t.hideturtle()

```


9.30 把以下代码片段插入 `if/elif` 中:

```
elif cmd == 'G':
    aTurtle.up()
    aTurtle.forward(distance)
    aTurtle.down()
```

第 10 章

```
10.1 class Planet:
    def __init__(self, iName, iRad, iM, iDist, iMoons):

        self.__name = iName
        self.__radius = iRad
        self.__mass = iM
        self.__distance = iDist
        self.__numMoons = iMoons
```

10.4 和 10.5

```
class Sentence:
    def __init__(self, initialString):
        self.__theString = initialString

    def getSentence(self):
        return self.__theString

    def getWords(self):
        return self.__theString.split()

    def getLength(self):
        return len(self.__theString)

    def getNumWords(self):
        return len(self.__theString.split())
```

```
10.8 def setMoons(self, numMoons):
    self.__numMoons = numMoons;
```

```
10.11 def addPunc(self, punc):
    self.__theString = self.__theString + punc
```

```
10.14 def __getitem__(self, index):
    return self.__theString.split()[index]
```

```
10.18 def __add__(self, otherSentence):
    newString = self.__theString + ' ' \
                + otherSentence.getSentence()
    return Sentence(newString)
```

10.19 和 10.20

```
import random
class Die:
    def __init__(self, iNumSides):
        self.__numSides = iNumSides
        self.__currRoll = self.roll()
    def roll(self):
        self.__currRoll = random.randint(1, self.__numSides)
        return self.__currRoll
    def getCurrRoll(self):
        return self.__currRoll
    def __str__(self):
        return str(self.__currRoll)
    def __eq__(self, otherDie):
        return self.getCurrRoll() == otherDie.getCurrRoll()
    def __ne__(self, otherDie):
        return self.getCurrRoll() != otherDie.getCurrRoll()
    def __lt__(self, otherDie):
        return self.getCurrRoll() < otherDie.getCurrRoll()
    def __le__(self, otherDie):
```

```

        return self.getCurrRoll() <= otherDie.getCurrRoll()
    def __gt__(self, otherDie):
        return self.getCurrRoll() > otherDie.getCurrRoll()
    def __ge__(self, otherDie):
        return self.getCurrRoll() >= otherDie.getCurrRoll()

```

示例:

```

>>> d1 = Die(6)
>>> d2 = Die(6)
>>> d1.roll()
6
>>> print(d1)
6
>>> d2.roll()
3
>>> print(d2)
3
>>> d1 == d2
False
>>> d1 != d2
True
>>> d1 < d2
False
>>> d1 <= d2
False
>>> d1 > d2
True
>>> d1 >= d2
True

```

10.23 和 10.24

```

class BankAccount:
    def __init__(self, iId):
        self.__idNum = iId
        self.__balance = 0
    def deposit(self, amount):
        self.__balance = self.__balance + amount
    def withdraw(self, amount):
        self.__balance = self.__balance - amount
    def getBalance(self):
        return self.__balance
    def transferTo(self, otherAcct, amount):
        self.withdraw(amount)
        otherAcct.deposit(amount)

```

10.28 class Sun:

```

    def __init__(self, iName, iRad, iM, iTemp):
        self.__name = iName
        self.__radius = iRad
        self.__mass = iM
        self.__temp = iTemp
    def getMass(self):
        return self.__mass
    def getRadius(self):
        return self.__radius
    def getTemperature(self):
        return self.__temp
    def getVolume(self):
        import math
        v = 4/3 * math.pi * self.__radius**3
        return v
    def getSurfaceArea(self):
        import math
        sa = 4 * math.pi * self.__radius**2

```

```

        return sa
    def getDensity(self):
        d = self.__mass / self.getVolume()
        return d
    def __str__(self):
        return self.__name
    def setName(self, newName):
        self.__name = newName
    def setRadius(self, newRad):
        self.__radius = newRad
10.31 def getTotalMass(self):
        totalMass = self.__theSun.getMass()
        for aPlanet in self.__planets:
            totalMass = totalMass + aPlanet.getMass()
        return totalMass
10.33 def getNearest(self):
        nearest = self.__planets[0]
        for aPlanet in self.__planets:
            if aPlanet.getDistance() < nearest.getDistance():
                nearest = aPlanet
        return nearest
    def getFarthest(self):
        farthest = self.__planets[0]
        for aPlanet in self.__planets:
            if aPlanet.getDistance() > farthest.getDistance():
                farthest = aPlanet
        return farthest

```

调用名为 ss 的 SolarSystem 的上述方法：

```

>>> print(ss.getNearest())
>>> print(ss.getFarthest())

```

第 11 章

```

11.1 for aCol in range(self.__maxX):
        col = []
        for aRow in range(self.__maxY):
            col.append(None)
        self.__grid.append(col)
11.7 def tryToMove(self):
        offsetList = [(-1,1), (0,1), (1,1),
                      (-1,0),          (1,0),
                      (-1,-1), (0,-1), (1,-1)]
        tryCount = 0
        goodMove = False
        while tryCount < 4 and not goodMove:
            randomOffsetIndex = random.randrange(len(offsetList))
            randomOffset = offsetList[randomOffsetIndex]
            nextX = self.__xPos + randomOffset[0]
            nextY = self.__yPos + randomOffset[1]
            while not (0 <= nextX < self.__world.getMaxX() and \
                      0 <= nextY < self.__world.getMaxY()):
                randomOffsetIndex = \
                    random.randrange(len(offsetList))
                randomOffset = offsetList[randomOffsetIndex]
                nextX = self.__xPos + randomOffset[0]
                nextY = self.__yPos + randomOffset[1]

            if self.__world.emptyLocation(nextX, nextY):
                self.move(nextX, nextY)
                goodMove = True

```

```

        else:
            tryCount = tryCount + 1
11.9 class Bear:
    def __init__(self):
        self.__turtle = turtle.Turtle()
        self.__turtle.up()
        self.__turtle.hideturtle()
        self.__turtle.shape("Bear.gif")

        self.__xPos = 0
        self.__yPos = 0
        self.__world = None

        self.__starveTick = 0
        self.__breedTick = 0
        self.__starveThreshold = 10
        self.__breedThreshold = 8

    # 其他方法

    def liveALittle(self):
        self.__breedTick = self.__breedTick + 1
        if self.__breedTick >= self.__breedThreshold:
            self.tryToBreed()

        self.tryToEat()

        if self.__starveTick == self.__starveThreshold:
            self.__world.delThing(self)
        else:
            self.tryToMove()

```

第 12 章

```

12.2 class Point(GeometricObject):
    def __init__(self, x, y):
        super().__init__()
        self.__pos = (x, y)
    def getCoord(self):
        return self.__pos
    def getX(self):
        return self.__pos[0]
    def getY(self):
        return self.__pos[1]
    def _draw(self, turtle):
        turtle.goto(self.__pos[0], self.__pos[1])
        turtle.dot(self.getWidth(), self.getColor())

```

```

12.4 >>> from draw import *
>>> myCanvas = Canvas(800, 600)
>>> x = 0
>>> for i in range(360):
    y = 100 * math.sin(math.radians(i))
    myPoint = Point(x, y)
    myPoint.setWidth(3)
    myPoint.setColor('blue')
    myCanvas.draw(myPoint)
    x = x + 1

```

12.7 使用图 12-5，查找顺序为箭头 1、2、3 和 4。

12.10 把以下代码添加到 Canvas 类：

```
def undraw(self, gObject):
    self.__visibleObjects.remove(gObject)
    gObject.setVisible(False)
    self.drawAll()
```

12.13 class Polygon(Shape):

```
def __init__(self, pList):
    super().__init__()
    self.__corners = pList
def _draw(self, aTurtle):
    aTurtle.color(self.getColor())
    aTurtle.width(self.getWidth())
    aTurtle.goto(self.__corners[0].getCoord())
    aTurtle.down()
    if self.getFillColor() != None:
        aTurtle.fillcolor(self.getFillColor())
        aTurtle.begin_fill()
    for cIndex in range(1, len(self.__corners)):
        aTurtle.goto(self.__corners[cIndex].getCoord())
    aTurtle.goto(self.__corners[0].getCoord())
    if self.getFillColor() != None:
        aTurtle.end_fill()

def addPoint(self, aPoint):
    self.__corners.append(aPoint)
def getPoints(self):
    return self.__corners
```

12.14 Polygon 的每个子类都应该创建一个 Point 对象列表并发送给 Polygon 构造方法。例如，下面是 Rectangle 类：

```
class Rectangle(Polygon):
    def __init__(self, lowerLeft, upperRight):
        self.__cornerList = []
        self.__cornerList.append(lowerLeft)
        self.__cornerList.append(Point(lowerLeft.getX(), \
                                         upperRight.getY()))
        self.__cornerList.append(upperRight)
        self.__cornerList.append(Point(upperRight.getX(), \
                                         lowerLeft.getY()))
        super().__init__(self.__cornerList)
```

第 13 章

13.4 (13.5 类似)

```
>>> from threading import Thread
>>> import time
>>> class EventHandler(Thread):
    def __init__(self):
        super().__init__()
        self.__queue = []
        self.__eventKeeper = {}

    def addEvent(self, eventName, eventData):
        self.__queue.append((eventName, eventData))

    def registerCallback(self, event, func):
        self.__eventKeeper[event] = func

    def run(self):
        while (True):
            if len(self.__queue) > 0:
```

```

        nextEvent, eventData = self.__queue.pop(0)
        callBack = self.__eventKeeper[nextEvent]
        callBack(eventData)      # 传递参数给函数
    else:
        time.sleep(1)            # 休眠 1 秒钟

>>> def myKey(key):
>>>     print('The {0} key was pressed'.format(key))
>>> eh = EventHandler()
>>> eh.addEvent('key', 'j')
>>> eh.registerCallback('key', myKey)
>>> eh.start()
The j key was pressed

```

13.7 class Etch(Turtle):

```

    def __init__(self):
        super().__init__()
        self.__screen = self.getscreen()
        self.color('blue')
        self.pensize(2)
        self.speed(0)
        self.__distance = 5
        self.__turn = 10
        self.__screen.onkey(self.fwd, "Up")
        self.__screen.onkey(self.bkwd, "Down")
        self.__screen.onkey(self.left10, "Left")
        self.__screen.onkey(self.right10, "Right")
        self.__screen.onkey(self.quit, "q")
        self.__screen.onkey(self.tailUp, "u")
        self.__screen.onkey(self.tailDown, "d")
        self.__screen.listen()
        self.main()

    def fwd(self):
        self.forward(self.__distance)

    def bkwd(self):
        self.backward(self.__distance)

    def left10(self):

        self.left(self.__turn)

    def right10(self):
        self.right(self.__turn)

    def quit(self):
        self.screen.bye()

    def tailUp(self):
        self.up()

    def tailDown(self):
        self.down()

    def main(self):
        mainloop()

```

13.11 在 placeTurtle 方法中, 修改最后 2 行代码为:

```

if self.__numTurtles >= self.__maxTurtles:
    self.__bigTscreen.onclick(self.moveTo)

```

然后在类中添加以下方法:

```
def moveTo(self, x, y):
    for t in self.__turtleList:
        t.setheading(t.towards(x, y))
        t.forward(10)
```

13.19 按以下步骤修改 AnimatedTurtle 方法:

1) 修改 __init__ 方法:

(a) 添加一个实例变量 __moving, 并初始化为 True:

```
self.__moving = True
```

(b) 为 onclick 事件添加一个回调函数:

```
self.onclick(self.stopGo)
```

2) 添加以下方法:

```
def stopGo(self, x, y):
    for t in AnimatedTurtle.__allTurtles:
        if t.distance(x, y) < 10:
            if t.__moving:
                t.__moving = False
            else:
                t.__moving = True
```

3) 修改 moveOneStep 方法如下:

```
def moveOneStep(self):
    if self.__moving:
        self.computeNewHeading()
        self.forward(5)
        self.checkCollisions()
    self.__scr.ontimer(self.moveOneStep, 100)
```

13.21 (a) 添加 ScoreKeeper 类:

```
class ScoreKeeper(Turtle):
    gameScore = 0    # 静态变量

    @staticmethod
    def alterScore(amount):
        ScoreKeeper.gameScore = ScoreKeeper.gameScore + amount

    def __init__(self, scoreX, scoreY):
        super().__init__()
        self.hideturtle()
        self.up()
        self.goto(scoreX, scoreY)
        self.down()
        self.__oldScore = -1    # 强制首先显示
        self.writeScore()

    def writeScore(self):
        if ScoreKeeper.gameScore != self.__oldScore:
            self.clear()
            self.write('Score: ' + str(ScoreKeeper.gameScore), \
                       move = False)
            self.__oldScore = ScoreKeeper.gameScore
```

(b) 修改 DroneInvaders 类:

```
class DroneInvaders:
    def __init__(self, xMin, xMax, yMin, yMax):
        self.__xMin = xMin
```

```

self.__xMax = xMax
self.__yMin = yMin
self.__yMax = yMax

def play(self):
    self.__mainWin = LaserCannon(self.__xMin, self.__xMax, \
                                   self.__yMin, self.__yMax).getscreen()
    self.__mainWin.bgcolor('light green')
    self.__mainWin.setworldcoordinates(self.__xMin, \
                                         self.__yMin, self.__xMax, self.__yMax)
    # 创建计分器 scoreKeeper
    self.__scoreKeeper = ScoreKeeper(self.__xMin + 50, \
                                       self.__yMax - 50)

    self.__mainWin.ontimer(self.addDrone, 200)
    self.__mainWin.listen()
    mainloop()

def addDrone(self):
    if len(Drone.getDrones()) < 7:
        Drone(1, self.__xMin, self.__xMax, \
              self.__yMin, self.__yMax)
    self.__scoreKeeper.writeScore() # 更新计分
    self.__mainWin.ontimer(self.addDrone, 200)

```

(c) 修改 Bomb 类的 move 方法，击中 Drone 时加 10 分：

```

def move(self):
    exploded = False
    self.forward(self.getSpeed())
    for d in Drone.getDrones():
        if self.distance(d) < 5:
            d.remove()
            ScoreKeeper.alterScore(10)
            exploded = True

    if self.outOfBounds() or exploded:
        self.remove()
    else:
        self.getscreen().ontimer(self.move, 100)

```

(d) 修改 Drone 类的 move 方法，当 Drone 到达地面（出界）时减 10 分。

```

def move(self):
    self.forward(self.getSpeed())
    if self.outOfBounds():
        self.remove()
        ScoreKeeper.alterScore(-10)
    else:
        self.getscreen().ontimer(self.move, 200)

```